

PlatformALife Core System Scripts

This document contains the foundational C# scripts for the **PlatformALife** project. These scripts should be saved into the project structure as specified below.

1. Deed.Data.cs

Location: scripts/data/Deed.Data.cs

```
using System;
using System.Collections.Generic;
using System.Text.Json;

namespace PlatformALife.Data
{
    public class FurnitureInstance
    {
        public int TypeID { get; set; }
        public float X { get; set; }
        public float Y { get; set; }
        public float Rotation { get; set; }
    }

    public class DeedData
    {
        public string OwnerID { get; set; }
        public int MapCount { get; set; }
        public List<string> UpgradesList { get; set; }
        public List<FurnitureInstance> FurnitureData { get; set; }

        public string Serialize() => JsonSerializer.Serialize(this);
        public static DeedData Deserialize(string json) =>
        JsonSerializer.Deserialize<DeedData>(json);
    }
}
```

2. DP.Manager.cs

Location: scripts/systems/DP.Manager.cs

```
using Godot;
using System;

namespace PlatformALife.Systems
{
    public partial class DPManager : Node
```

```

{
    [Export] public int DailyTaskCompletion = 0;
    public bool IsXpBuffActive = false;
    private DateTime _lastResetTime;

    public override void _Ready()
    {
        _lastResetTime = DateTime.UtcNow.Date;
    }

    public void CheckForReset()
    {
        if (DateTime.UtcNow.Date > _lastResetTime)
        {
            DailyTaskCompletion = 0;
            IsXpBuffActive = false;
            _lastResetTime = DateTime.UtcNow.Date;
        }
    }
}

```

3. WorldGenerator.cs

Location: scripts/world/WorldGenerator.cs

```

using Godot;
using System;

namespace PlatformALife.World
{
    public partial class WorldGenerator : Node2D
    {
        private const int ERROR_ATLAS_ID = 999;

        public void ValidateAndSetCell(TileMapLayer layer, Vector2I
coords, int atlasId, Vector2I atlasCoords, string biomeContext)
        {
            var tileSet = layer.TileSet;
            if (atlasId < 0 || atlasId >= tileSet.GetSourceCount())
return;

            var source = tileSet.GetSource(atlasId) as
TileSetAtlasSource;
            if (source != null && source.HasTile(atlasCoords))
            {
                layer.SetCell(coords, atlasId, atlasCoords);
            }
        }
    }
}

```

```
        }
        else
        {
            GD.PushWarning($"[WorldGenerator] Invalid
{atlasCoords} in {biomeContext}.");
            layer.SetCell(coords, ERROR_ATLAS_ID, new Vector2I(0,
0));
        }
    }
}
}
```